

Praktikum 9 - Matakuliah Pilihan 1 (Web)

Program Studi: Teknik Informatika

Nama: Salsa Putri Ajriyanti

Nim: 3312401043

Kelas: IF 3A Web Pagi

Lakukan praktikum dibawah ini, dan buat screenshot untuk pembuktian mengerjakan setiap poin dengan mengisi tabel dibawah, kemudian tunjukan hasil akhir dari men-share repository github yang telah dibuat.

A. Menyediakan API Enpoints

1. Lanjutkan Project Praktikum 8, dengan menggunakan file yang sama (copy)
2. Tambahkan pada database, sebuah tabel produk Isi tabel produk seperti pada tabel berikut: (Buat 10 item)

id	nama	deskripsi	harga	foto
1.	Indomie Goreng		2500	images/miegoreng.jpg
2.	...	...	...	...
10.	...	...	..	...

3. Seperti pada perintah praktikum 8 buatkan beberapa endpoints

GET : localhost:8001/api/products/

GET : localhost:8001/api/products/:id

POST : localhost:8001/api/products/

PUT : localhost:8001/api/products/:id

DELETE: localhost:8001/api/products/:id

4. Pastikan memiliki file dengan struktur sebagai berikut.

controllers/[products.controller.js](#)

routes/[products.routes.js](#)

models/[products.model.js](#)

5. Test API dengan menggunakan **POSTMAN**

B. Menambahkan Proteksi API (Simple - Bearer Method)

1. Buat sebuah folder bernama `middlewares`, kemudian buat didalamnya sebuah file dengan nama `auth.middleware.js` dengan kode program seperti dibawah ini

```
export const authBearer = (req, res, next) => {  
  const authHeader = req.headers.authorization;  
  
  // Tidak ada Authorization  
  if (!authHeader) {  
    return res.status(401).json({ message: "No authorization header" });  
  }  
  
  // Harus Bearer  
  if (!authHeader.startsWith("Bearer ")) {  
    return res.status(401).json({ message: "Bearer token required" });  
  }  
  
  // Ambil token  
  const token = authHeader.split(" ")[1];  
  
  // Token yang benar (misal hardcode)  
  const VALID_TOKEN = "12345TOKENRAHASIA";  
  
  if (token !== VALID_TOKEN) {  
    return res.status(403).json({ message: "Invalid token" });  
  }  
  
  next();  
};
```

2. Pada file `product.routes.js` panggil `auth.middleware.js` `import { authBearer } from "../middleware/auth.middleware.js"`
3. Lalu tambahkan `authBearer` ke endpoints yang perlu di proteksi
`router.post("/", authBearer, createProduct); router.put("/:id", authBearer, updateProduct); router.delete("/:id", authBearer, deleteProduct);`

- Gunakan POSTMAN untuk akses 3 endpoints ini dengan menambahkan bearer
12345TOKENRAHASIA

F. Github + Visual Code

- Buat proyek di Github dengan nama **Latihan9**

git init

git add

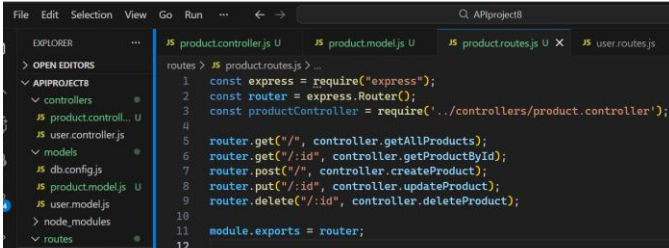
.

git commit -m "first commit" git branch -M main git remote add

origin https://github.com/agunghakase/Latihan9.git git push -u

origin main

Hasil Pengerjaan

No.	Instruksi	Screenshot	Kendala/Saran
A.	Menyediakan API points		
1.	Tambahkan pada database, sebuah tabel produk Isi tabel produk seperti pada tabel berikut: (Buat 10 item)		
2.	Buatkan beberapa endpoints dan file route, controller, serta model produk seperti praktikum 8		

3.

Test API dengan POSTMAN

The screenshot displays two Postman requests. The first is a GET request to `localhost:8001/api/products/` which returns a JSON array of three product objects. The second is a POST request to `localhost:8001/api/products` with a JSON body representing a new product, which returns a JSON object with an assigned ID.

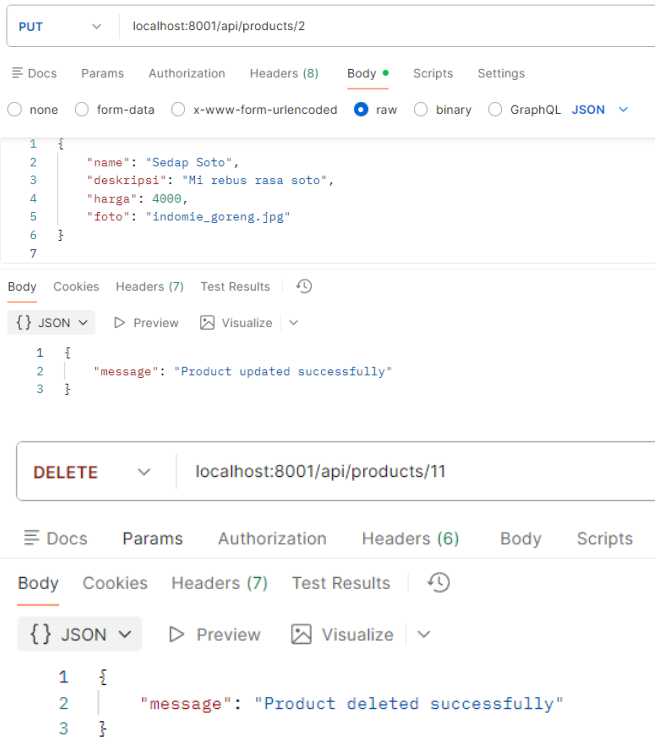
```
GET localhost:8001/api/products/

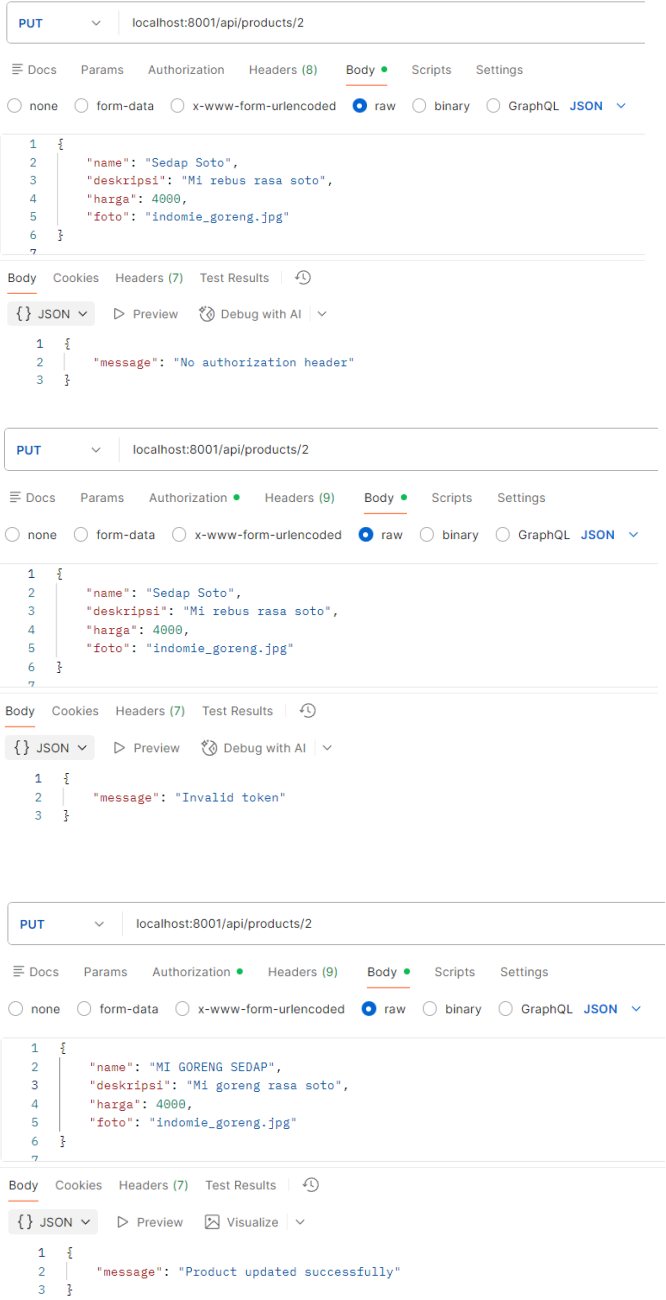
Body
JSON
1 [
2   {
3     "id": 1,
4     "name": "Indomie Goreng",
5     "deskripsi": "Mi instan goreng favorit dengan cita rasa gurih pedas manis.",
6     "harga": 3500,
7     "foto": "indomie_goreng.jpg",
8     "created_at": "2025-11-19T03:12:56.000Z",
9     "updated_at": "2025-11-19T03:12:56.000Z"
10  },
11  {
12    "id": 2,
13    "name": "Mie Sedap Ayam Bawang",
14    "deskripsi": "Mi kuah dengan rasa ayam bawang yang gurih dan nikmat.",
15    "harga": 3200,
16    "foto": "sedap_ayam_bawang.jpg",
17    "created_at": "2025-11-19T03:12:56.000Z",
18    "updated_at": "2025-11-19T03:12:56.000Z"
19  },
20  {
21    "id": 3,
22    "name": "Kopi Kapal Api",
23    "deskripsi": "Kopi hitam mantap dengan aroma khas, cocok untuk pagi hari.",
24    "harga": 1500,
25    "foto": "kapal_api.jpg",
26    "created_at": "2025-11-19T03:12:56.000Z",
27    "updated_at": "2025-11-19T03:12:56.000Z"
28  }
29 ]
```

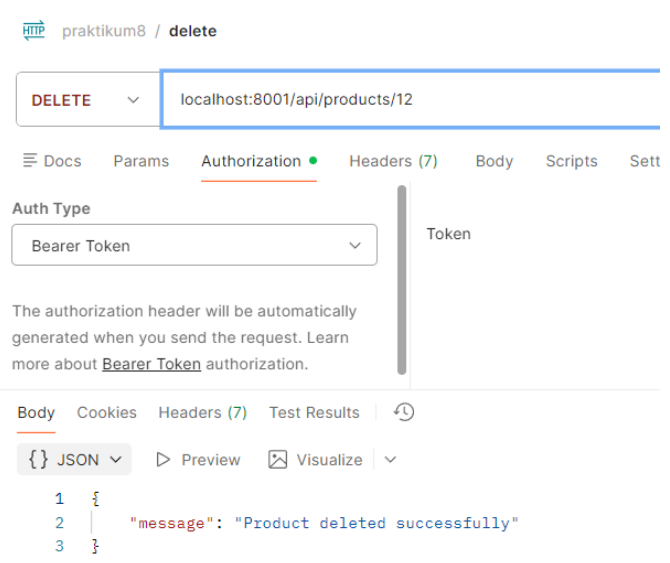
```
POST localhost:8001/api/products

Body
raw
1 {
2   "name": "Indomie Goreng",
3   "deskripsi": "mie instan enak",
4   "harga": 3500,
5   "foto": "indomie.jpg"
6 }
7

Body
JSON
1 {
2   "id": 11,
3   "name": "Indomie Goreng",
4   "deskripsi": "mie instan enak",
5   "harga": 3500,
6   "foto": "indomie.jpg"
7 }
```

		 <pre> PUT localhost:8001/api/products/2 { "name": "Sedap Soto", "deskripsi": "Mi rebus rasa soto", "harga": 4000, "foto": "indomie_goreng.jpg" } { "message": "Product updated successfully" } DELETE localhost:8001/api/products/11 { "message": "Product deleted successfully" } </pre>	
B.	Menambahkan Proteksi API (Simple - Bearer Method)		
1.	Buat sebuah folder bernama middlewares, kemudian buat didalamnya sebuah file dengan nama auth.middleware.js dengan kode program seperti dibawah	<pre> middlewares > JS auth.middleware.js > authBearer 1 export const authBearer = (req, res, next) => { 2 const authHeader = req.headers.authorization; 3 4 //tidak ada authorization 5 if (!authHeader) { 6 return res.status(401).json({ message: 'No authorization header' }); 7 } 8 9 //harus bearer 10 if (!authHeader.startsWith("Bearer ")) { 11 return res.status(401).json({ message: 'Bearer token required' }); 12 } 13 14 //ambil token 15 const token = authHeader.split(" ")[1]; 16 17 //token yang benar (misal hardcode) 18 const VALID_TOKEN = "12345TOKENRAHASIA"; 19 20 if (token !== VALID_TOKEN) { 21 return res.status(403).json({ message: 'Invalid token' }); 22 } 23 24 next(); 25 } </pre>	
2.	Pada file product.routes.js panggil auth.middleware.js <code>import { authBearer } from "../middleware/auth. middleware.js"</code>	<pre> routes > JS product.routes.js > ... 1 import express from 'express'; 2 import { authBearer } from '../middleware/auth.middleware.js'; </pre>	

3.	Lalu tambahkan authBearer ke endpoints yang perlu di proteksi router.post("/", authBearer, createProduct); router.put("/:id", authBearer, updateProduct); router.delete("/:id", authBearer, deleteProduct);	<pre>routes > \$S product/routes.js > ... 1 import express from 'express'; 2 import { authBearer } from '../middlewares/auth.middleware.js'; 3 import * as productController from '../controllers/product.controller.js'; 4 5 const router = express.Router(); 6 7 router.get("/", productController.getAllProducts); 8 router.get("/:id", productController.getProductById); 9 router.post("/", authBearer, productController.createProduct); 10 router.put("/:id", authBearer, productController.updateProduct); 11 router.delete("/:id", authBearer, productController.deleteProduct); 12 13 export default router;</pre>	
4.	Gunakan POSTMAN untuk akses 3 endpoints ini dengan menambahkan bearer 12345TOKENRAHASIA	 The first screenshot shows a PUT request to localhost:8001/api/products/2 with a JSON body: { "name": "Sedap Soto", "deskripsi": "Mi rebus rasa soto", "harga": 4000, "foto": "indomie_goreng.jpg" }. The response is { "message": "No authorization header" }. The second screenshot shows the same PUT request with the Authorization header set to Bearer 12345TOKENRAHASIA. The response is { "message": "Invalid token" }. The third screenshot shows the same PUT request with the Authorization header set to Bearer 12345TOKENRAHASIA. The response is { "message": "Product updated successfully" }.	Pertama menggunakan type commonjs, trs Ganti menjadi type module, dimana semua kode harus di Ganti sesuai dengan penulisan module tidak boleh campur

			
C.	Commit github		
1.	git commit	